sometimes make them less useful for more complex user interfaces, such as modern JavaScript one-page applications.

If your application does rely heavily on AJAX and JavaScript, then PhantomJS provides significantly more reliable browser emulation than HtmlUnit, including the features that you'd expect of a real browser, such as good support for dynamic asynchronous behavior and the ability to capture screenshots. PhantomJS is used by a number of web-testing libraries, including WebDriver. Although it's not a great deal faster than a real browser for individual tests, it tends to be more tolerant of parallel testing.

It's important to note that headless browsers don't have exactly the same behavior, or render in exactly the same way, as real browsers. HtmlUnit uses the Rhino JavaScript implementation, which isn't used by a real browser. PhantomJS uses WebKit, which may have different behavior than Firefox or Internet Explorer. And if your application contains important business logic in the client, you may want to verify that it also works correctly across different browsers, including a range of real ones.

8.1.3 How much web testing do you really need?

Web tests clearly have their uses. But you rarely need to test every aspect of a system using web tests, and doing so is generally not a good idea. In fact, in a typical BDD project, a significant proportion of automated acceptance tests will be implemented as non-web tests (see figure 8.3). These non-UI tests can take many forms, as you'll see in chapter 9, including what would traditionally be classed as integration or unit tests. Many automated acceptance criteria, particularly those related to business rules or calculations, are more effectively done directly using the application code rather than via the user interface, as non-web tests can test specific business rules more quickly and more precisely than an end-to-end web test.

Of course, it can be tricky to know whether to implement an acceptance test as a web test or a non-web test. You only need a web test for two things:

- Illustrating the user's journey through the system
- Illustrating how a business rule is represented in the user interface

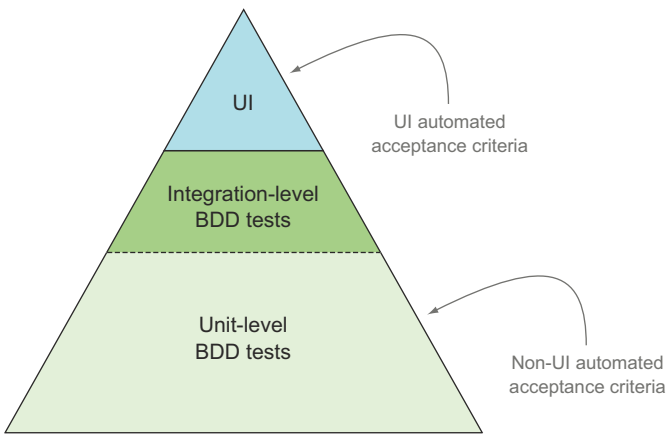


Figure 8.3 A typical BDD project will have many more non-UI automated acceptance tests than UI ones.